

HW07: Secure Peer-to-Peer Roll a Dice

Simone Nol 
1940213

December 6, 2025

Abstract

This report presents the design and implementation of a secure Peer-to-Peer (P2P) protocol for playing "Roll a Dice", with multiple dices. To ensure fairness and prevent cheating in a trustless environment, the protocol implements a **Bit Commitment Scheme** using SHA-256 hashing to hide moves, strict **Timeout Mechanisms** to handle disconnection attempts by losing players, and a random function with a well crafted seed to verify each dice rolled by Bob and Alice. The system is deployed using Docker containers to simulate distinct network nodes.

Contents

1	Introduction	2
2	Protocol Design: The Commitment Scheme	2
2.1	Phase 1: Commitment	2
2.2	Phase 2: The Game (Bob's Move)	2
2.3	Phase 3: Reveal and Verification	3
3	Security Implementation Details	3
3.1	Preventing Abort Attacks (Timeout Strategy)	3
4	Infrastructure and Virtualization	3
4.1	Docker Build Environment	3
4.2	Docker Compose Configuration	3
4.3	Network Simulation	4
5	Execution Logs	4
5.1	Scenario A: Honest Game	4
5.2	Scenario B: Abort Attack (Cheating Attempt)	5
6	Conclusion	5
7	Code	6
7.1	tcp_json.py	6
7.2	bob.py	6
7.3	alice.py.py	9

1 Introduction

The objective of this assignment is to allow two remote entities, Alice and Bob, to play "Roll a Dice" over a network. Unlike a Client-Server architecture where a central authority guarantees fairness, this solution adopts a **Peer-to-Peer** approach.

In a P2P scenario, neither player trusts the other. This introduces specific security challenges:

- **The Pre-knowledge Attack:** If Alice sends her result first in cleartext, Bob can see it and choose the winning value.
- **The Always winning Attack:** Both Alice and Bob can decide to always return the maximum value (six, for example).
- **The Abort Attack (Sore Loser):** If Alice waits for Bob to reveal his move and realizes she has lost, she might disconnect to avoid the loss being recorded.

2 Protocol Design: The Commitment Scheme

To address the "Pre-knowledge Attack" and "The Always winning Attack", it has been implemented a cryptographic **Commitment Scheme**. This ensures that a player sends a result and "locks" it before revealing it, making it impossible to change later. In particular, it has decided to implement the Commitment Scheme not to the dices values but to A and B , which are very high values generated respectively by Alice and Bob. When Bob and Alice know that no one cheated, they sum A and B to get $A + B$, which is a value that will be used as a seed in a pseudorandom function. With that function, it is possible to obtain each Alice's and Bob's values. In this way, no one can change or lie about the result. The protocol proceeds in three phases:

2.1 Phase 1: Commitment

Alice (the Initiator) sends her 256 bit result M_A and generates a high-entropy random nonce N_A (128-bit). She calculates the commitment Hash H :

$$H = \text{SHA-256}(M_A || N_A)$$

Alice sends H to Bob. Bob now knows Alice has played, but cannot know what she played.

2.2 Phase 2: The Game (Bob's Move)

Bob (the Peer/Listener), upon receiving H , sends his 256 bit result M_B in cleartext to Alice.

- Bob cannot cheat because he doesn't know M_A .
- Alice cannot change M_A because it must match the hash H she already sent.

2.3 Phase 3: Reveal and Verification

Upon receiving M_B , Alice sends the pair (M_A, N_A) to Bob. Bob verifies the game validity by re-computing the hash:

$$H_{check} = \text{SHA-256}(M_A || N_A)$$

If $H_{check} == H$, the game is valid, and the winner is determined.

3 Security Implementation Details

3.1 Preventing Abort Attacks (Timeout Strategy)

The "Abort Attack" occurs when Alice receives Bob's move, calculates the result locally, sees she has lost, and refuses to send the Reveal message (Phase 3). To solve this, Bob implements a strict **Socket Timeout**. This rule ensures that disconnecting is equivalent to losing.

4 Infrastructure and Virtualization

To meet the requirement of implementing the protocol on distinct virtual machines and simulating a realistic network environment, it has been adopted a **Container-based architecture** using Docker.

This approach ensures process isolation and provides a separate network stack for each actor (Alice and Bob), effectively simulating two distinct machines connected via a local network.

4.1 Docker Build Environment

The application environment for each service (`alice`, and `bob`) is defined by a single Dockerfile.

```
1 FROM python:3.10-slim
2 WORKDIR /app
3 COPY . /app
```

Listing 1: Dockerfile

4.2 Docker Compose Configuration

The orchestration of the environment is managed via `docker-compose`, which defines three services:

- **Bob:** It is accessible to alice via the hostname `bob`.
- **Alice:** This executes the client logic.

The following is the configuration used to deploy the environment:

```

1 services:
2   bob:
3     build: .
4     command: python bob.py
5     ports:
6       - "8080:8080"
7     hostname: bob
8     restart: always
9   alice:
10    build: .
11    command: python alice.py
12    depends_on:
13      - bob
14    stdin_open: true
15    tty: true
16    restart: always

```

Listing 2: Docker Compose Configuration

4.3 Network Simulation

Docker automatically creates a virtual bridge network (default subnet). Within this network, **Service Discovery** is handled via internal DNS. Instead of hardcoding IP addresses (e.g., 192.168.1.x), the `alice.py` connects to the server using the hostname `bob`. This abstraction mimics a real-world DNS resolution scenario and makes the code environment-agnostic.

- **Server Address:** `bob` (Resolved automatically by Docker DNS)
- **Port:** 8080 (Mapped for internal communication)

5 Execution Logs

Below are the execution traces proving the fairness of the protocol.

5.1 Scenario A: Honest Game

Alice sends her value (hidden), Bob also sends his value. Alice sees that she lost and decides to reveal her value anyways. Bob wins.

```

1 # Alice's Output
2 [Alice] I'm going to meet Bob...
3 [Alice] Arrived to Bob.
4 [Alice] Waiting for message... w Enable Watch
5 [Alice] Sent my move
   (98112930973083730968033582691242975596431905903910044920173098802759775317728)
   without nonce (b95cd137ef6329503b9ff12e55ce9dee), I want to be sure that Bob cannot
   cheat
6 [Alice] Received Bob's move:
   43515266722048216225652609136380639278543702059628004125693271560873580804461
7 Dice Alice: [2, 2, 4, 5, 2] (Tot: 15)
8 Dice Bob:   [1, 6, 6, 6, 4] (Tot: 23)
9 [Alice] Bob wins...
10 [Alice] The winner is: Bob
11 [Alice] Should I send the nonce to Bob? I can escape...(yes/no)
12 yes
13 [alice] Sent nonce to Bob.
14

```

```

15 # Bob's output
16 [BOB] Waiting for Alice to arrive...
17 [BOB] Alice has arrived.
18 [BOB] Sent game start message to Alice.
19 [BOB] Waiting for message...
20 [BOB] Received Alice's commitment: 5339
    e74c068d8dcb842bcab1125395c5dd087b674c8a8e7befdd6cba882fa6ab
21 [BOB] My move is:
    43515266722048216225652609136380639278543702059628004125693271560873580804461
22 [BOB] Sent my move to Alice.
23 [BOB] Waiting 5 seconds for Alice to reveal nonce...
24 [BOB] Waiting for message...
25 [BOB] Received Alice's nonce: b95cd137ef6329503b9ff12e55ce9dee
26 [BOB] Received Alice's move:
    98112930973083730968033582691242975596431905903910044920173098802759775317728
27 [BOB] Verifying Alice's commitment...
28 [BOB] Alice move:
    98112930973083730968033582691242975596431905903910044920173098802759775317728
29 Dice Alice: [2, 2, 4, 5, 2] (Tot: 15)
30 Dice Bob: [1, 6, 6, 6, 4] (Tot: 23)
31 [Bob] I win!
32 [BOB] The winner is: Bob

```

Listing 3: Honest Game Log

5.2 Scenario B: Abort Attack (Cheating Attempt)

Alice plays Rock, Bob plays Paper. Alice sees she lost and kills the connection to avoid admitting defeat. Bob wins by timeout.

```

1 # Alice's output
2 [Alice] I'm going to meet Bob...
3 [Alice] Arrived to Bob.
4 [Alice] Waiting for message...
5 [Alice] Sent my move
    (114929153547024847646241497826326569467526481132505607128430182568850569367333)
    without nonce (99c939529b15ccbc389c9d0ae910db4e), I want to be sure that Bob cannot
    cheat
6 [Alice] Received Bob's move:
    31699689225616967437803000959810189260411192479989776580103091247462600916664
7 Dice Alice: [1, 5, 2, 1, 6] (Tot: 15)
8 Dice Bob: [3, 1, 3, 6, 3] (Tot: 16)
9 [Alice] Bob wins...
10 [Alice] The winner is: Bob
11 [Alice] Should I send the nonce to Bob? I can escape...(yes/no)
12 no
13 [alice] I chose not to send the nonce to Bob. I have to run!
14 [Alice] game over, connection closed.
15
16 # Bob's output
17 [BOB] Waiting for Alice to arrive...
18 [BOB] Received Alice's commitment: 91
    dffc23da82ce9d3c56a017084fba1a6b34b9e3d934e858ea1694da7f888fee
19 [BOB] My move is:
    31699689225616967437803000959810189260411192479989776580103091247462600916664
20 [BOB] Sent my move to Alice.
21 [BOB] Waiting 5 seconds for Alice to reveal nonce...
22 [BOB] Waiting for message...
23 [Bob] Alice ran away! I win by default!
24 [Bob] game over. Connection closed.

```

Listing 4: Cheating Attempt Log

6 Conclusion

The implemented Peer-to-Peer protocol successfully mitigates both information leakage (via Bit Commitment) and evasion tactics (via Timeouts). This shows that a fair game

can be conducted over a network without relying on a central server, provided that the protocol enforces strict rules on information exchange and timing.

7 Code

7.1 tcp_json.py

```
1 import json
2 import struct
3
4 def recvall(sock, n):
5     """Helper function to receive EXACTLY n bytes or die trying"""
6     data = b''
7     while len(data) < n:
8         packet = sock.recv(n - len(data))
9         if not packet:
10             return None # the connection was closed
11         data += packet
12     return data
13
14 def receive_json(sock):
15
16     header = recvall(sock, 4)
17     if not header:
18         return None # the connection was closed
19
20     msg_length = struct.unpack('>I', header)[0]
21
22     payload_bytes = recvall(sock, msg_length)
23
24     try:
25         packet_dict = json.loads(payload_bytes.decode('utf-8'))
26         return packet_dict
27     except json.JSONDecodeError:
28         print("ERROR: Malformed JSON received!")
29         return None
30
31 def send_json(sock, packet_dict):
32
33     json_str = json.dumps(packet_dict)
34
35     data_bytes = json_str.encode('utf-8')
36
37     msg_length = len(data_bytes)
38
39     header = struct.pack('>I', msg_length)
40
41     sock.sendall(header + data_bytes)
```

Listing 5: tcp_json.py code that wrappers sendall and recvall function for TCP sockets and manages JSON communications properly

7.2 bob.py

```
1 import socket
2 import secrets
3 import random
4 from hashlib import sha256
5 from tcp_json import send_json
6 from tcp_json import receive_json
7 from time import sleep
8
9 HOST = "0.0.0.0"
10 PORT = 8080
11
12 num_dices = 5
```

```

13 num_alice_wins = 0
14 num_bob_wins = 0
15
16 alice_commitment = ""
17 my_move = ""
18 alice_move = ""
19 alice_nonce = ""
20 alice_should_send_nonce = False
21
22 def determine_winner(alice_move, my_move):
23     final_seed = my_move + alice_move
24
25     random.seed(final_seed)
26
27     alice_dice = []
28     for _ in range(num_dices):
29         dado = random.randint(1, 6)
30         alice_dice.append(dado)
31
32     bob_dice = []
33     for _ in range(num_dices):
34         dado = random.randint(1, 6)
35         bob_dice.append(dado)
36
37     sum_alice = sum(alice_dice)
38     sum_bob = sum(bob_dice)
39
40     print(f"Dice Alice: {alice_dice} (Tot: {sum_alice})")
41     print(f"Dice Bob: {bob_dice} (Tot: {sum_bob})")
42
43     global num_alice_wins, num_bob_wins
44     if sum_alice > sum_bob:
45         print("[Bob] Alice wins...")
46         num_alice_wins += 1
47         return "Alice"
48     elif sum_bob > sum_alice:
49         print("[Bob] I win!")
50         num_bob_wins += 1
51         return "Bob"
52     else:
53         print("[Bob] It's a draw!")
54         return "Draw"
55
56 def handle_game_commitment(message, conn):
57     global alice_commitment
58     alice_commitment = message.get("value")
59     print(f"[BOB] Received Alice's commitment: {alice_commitment}")
60
61     global my_move
62     my_move = secrets.randbits(256)
63     print(f"[BOB] My move is: {str(my_move)}")
64
65     response = {
66         "type": "bob-move",
67         "value": my_move
68     }
69     send_json(conn, response)
70     global alice_should_send_nonce
71     alice_should_send_nonce = True
72     print(f"[BOB] Sent my move to Alice.")
73     conn.settimeout(5.0)
74     print("[BOB] Waiting 5 seconds for Alice to reveal nonce...")
75
76 def handle_reveal_nonce(message):
77     global alice_nonce, alice_should_send_nonce
78     if not alice_should_send_nonce:
79         print("[BOB] Unexpected nonce reveal from Alice!")
80         return False
81     alice_should_send_nonce = False
82     alice_nonce = message.get("value")
83     print(f"[BOB] Received Alice's nonce: {alice_nonce}")
84
85     global alice_move

```

```

86     alice_move = message.get("alice-move")
87     print(f"[BOB] Received Alice's move: {str(alice_move)}")
88
89     check = False
90     print("[BOB] Verifying Alice's commitment...")
91     if alice_move is not None:
92         commitment_check = sha256((str(alice_move) + alice_nonce).encode()).hexdigest()
93         if commitment_check == alice_commitment:
94             check = True
95
96     if check:
97         print(f"[BOB] Alice move: {alice_move}")
98     else:
99         print("[BOB] Could not determine Alice's move from the nonce!")
100        return False
101
102    winner = determine_winner(alice_move, my_move)
103    print(f"[BOB] The winner is: {winner}")
104    return False
105
106    def handle(conn):
107
108        while True:
109            try:
110                print(f"[BOB] Waiting for message...")
111                msg = receive_json(conn)
112
113                if not msg:
114                    if alice_nonce == "":
115                        print(f"[Bob] Alice ran away! I win by default!")
116                        # Client disconnected
117                        break
118
119                match msg.get("type"):
120                    case "game-commitment":
121                        if alice_should_send_nonce:
122                            print("[Bob] Unexpected game commitment from Alice! She didn't
123                            send the nonce yet. She must be cheating!")
124                            break
125                        handle_game_commitment(msg, conn)
126                    case "reveal-nonce":
127                        handle_reveal_nonce(msg)
128                    case _:
129                        print(f"[Bob] Unknown message type: {msg.get('type')}")
130
131            except socket.timeout:
132                print(f"[Bob] Timeout waiting for Alice's nonce. I win by default!")
133                break
134
135        conn.close()
136        print(f"[Bob] game over. Connection closed.")
137        print(f"[Bob] Total Alice wins: {num_alice_wins}, Total Bob wins: {num_bob_wins}")
138
139    def main():
140        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
141            s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
142            s.bind((HOST, PORT))
143            s.listen()
144            print("[BOB] Waiting for Alice to arrive...")
145            sleep(2)
146            conn, addr = s.accept()
147            print("[BOB] Alice has arrived.")
148            message = {
149                "type": "game",
150            }
151            send_json(conn, message)
152            print("[BOB] Sent game start message to Alice.")
153            handle(conn)
154
155    if __name__ == "__main__":

```


Listing 6: bob.py is the server side of this Peer-To-Peer connection

7.3 alice.py.py

```

1 import secrets
2 import socket
3 import random
4 from hashlib import sha256
5
6 from time import sleep
7 from tcp_json import send_json
8 from tcp_json import receive_json
9
10 HOST = 'bob'
11 PORT = 8080
12
13 num_dices = 5
14 num_alice_wins = 0
15 num_bob_wins = 0
16 count_games = 0
17
18 my_move = ""
19 my_nonce = ""
20 bob_move = ""
21
22 def determine_winner(my_move, bob_move):
23     final_seed = my_move + bob_move
24
25     random.seed(final_seed)
26
27     alice_dice = []
28     for _ in range(num_dices):
29         dado = random.randint(1, 6)
30         alice_dice.append(dado)
31
32     bob_dice = []
33     for _ in range(num_dices):
34         dado = random.randint(1, 6)
35         bob_dice.append(dado)
36
37     sum_alice = sum(alice_dice)
38     sum_bob = sum(bob_dice)
39
40     print(f"Dice Alice: {alice_dice} (Tot: {sum_alice})")
41     print(f"Dice Bob: {bob_dice} (Tot: {sum_bob})")
42
43     global num_alice_wins, num_bob_wins, count_games
44     count_games += 1
45
46     if sum_alice > sum_bob:
47         print("[Alice] I win!")
48         num_alice_wins += 1
49         return "Alice"
50     elif sum_bob > sum_alice:
51         print("[Alice] Bob wins...")
52         num_bob_wins += 1
53         return "Bob"
54     else:
55         print("[Alice] It's a draw!")
56         return "Draw"
57
58 def handle_bob_move(message, conn):
59     global bob_move
60     bob_move = message.get("value")
61     print(f"[Alice] Received Bob's move: {str(bob_move)}")
62
63     winner = determine_winner(my_move, bob_move)
64     print(f"[Alice] The winner is: {winner}")
65     if winner == "Bob":

```

```

66     print("[Alice] Should I send the nonce to Bob? I can escape...(yes/no)")
67     choice = input().strip().lower()
68     if choice == "yes":
69         #print(F"[alice][DEBUG] my_move: {my_move} my_nonce: {my_nonce}.")
70         response = {
71             "type": "reveal-nonce",
72             "value": my_nonce,
73             "alice-move": my_move
74         }
75         send_json(conn, response)
76         print("[alice] Sent nonce to Bob.")
77     else:
78         print("[alice] I chose not to send the nonce to Bob. I have to run!")
79 else:
80     response = {
81         "type": "reveal-nonce",
82         "value": my_nonce,
83         "alice-move": my_move
84     }
85     send_json(conn, response)
86     print("[alice] Sent nonce to Bob.")
87 return True
88
89
90 def game(message, conn):
91     global my_move, my_nonce
92     my_move = secrets.randbits(256)
93     my_nonce = secrets.token_hex(16)
94     commitment = sha256((str(my_move) + my_nonce).encode()).hexdigest()
95     message = {
96         "type": "game-commitment",
97         "value": commitment
98     }
99
100    send_json(conn, message)
101    print(f"[Alice] Sent my move ({str(my_move)}) without nonce ({my_nonce}), I want to
be sure that Bob cannot cheat")
102    return True
103
104
105 def handle(message, conn):
106     msg_type = message.get("type")
107     match msg_type:
108         case "game":
109             return game(message, conn)
110         case "bob-move":
111             return handle_bob_move(message, conn)
112
113
114 def main():
115
116     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as conn:
117         print("[Alice] I'm going to meet Bob...")
118         sleep(2)
119         conn.connect((HOST, PORT))
120         conn.settimeout(5.0)
121         print("[Alice] Arrived to Bob.")
122
123         while True and count_games < 5:
124             print("[Alice] Waiting for message...")
125             if count_games > 0:
126                 game({}, conn)
127
128             message = receive_json(conn)
129             if not message:
130                 print("[Alice] No message received, closing connection")
131                 break
132             #print(f"message received : {message}")
133             if handle(message, conn) == False:
134                 break
135             conn.close()
136             print("[Alice] game over, connection closed.")
137     print(f"[Alice] Total Alice wins: {num_alice_wins}, Total Bob wins: {num_bob_wins},

```

```
138     Total games played: {count_games}")
139
140 if __name__ == "__main__":
141     main()
```

Listing 7: alice.py is like the client side of the Peer-To-peer connection